UNIVERSITI TEKNOLOGI MALAYSIA

Faculty of Computing
Semester 2
Session 2025/2026

SECJ3483 - SECTION 01
WEB TECHNOLOGY

Group Lab Activity: Security Investigation and Fix

Project Title : Secure Person BMI Full-Stack Web Application

Name	Matrics No.
MUHAMMAD AIDIL HAIKAL BIN MAZALAN	A23CS5010
MUHAMMAD AIMAN DANISH BIN MUHAMMAD EKHSAN	A23CS0119
MUHAMMAD FAKHRUL RAZZI BIN MD NOOR	A23CS0128
MUHAMMAD RIFQI BIN RAZALI	A23CS0136

2. Security investigation summary

Test No	What I Tested	Expected Secure Behavior	Actual Result	Is It a Weakness?	Why?
1	Add BMI with invalid data	Backend should reject invalid data	Data is accepted and stored in the database.	Yes	Missing backend validation because frontend validation can be bypassed using ThunderClient.
2	Access another user's BMI record	Backend should deny access	The private BMI record data is returned.	Yes	Broken Object Level Authorization because data ownership is not verified after a user logs in.
3	Access staff route as normal user	Backend should deny access	The user is allowed to access and view the data.	Yes	Broken Function Level Authorization because there are no server-side role checks for staff features.
4	Access admin route as normal user	Backend should deny access	The user is allowed to access and view the data.	Yes	Broken Function Level Authorization because the route lacks server-side admin protection middleware.
5	Submit XSS payload in notes	Payload should display as text	The browser executes the script by showing the alert.	Yes	Stored Cross-Site Scripting because the frontend renders input unsafely using v-html.
6	Check API response	Password/hash should not be visible	The password value is visible in the payload.	Yes	Sensitive data exposure because the API blindly returns all columns including password_hash.
7	Try SQL Injection input	Input should be treated as data	The structural database query behaves unexpectedly.	Yes	SQL Injection because the application constructs database queries using unsafe string concatenation.
8	Try to update protected fields	Backend should reject or ignore protected fields	The fields (e.g., role, user_id) are accepted and modified	Yes	Mass assignment because the backend blindly executes updates on protected fields like user_id and role.

Test 1: Add BMI with invalid data

- Did the backend accept the data?

Yes, it saved the negative and empty values directly into the database.

- Did the frontend validation stop you?

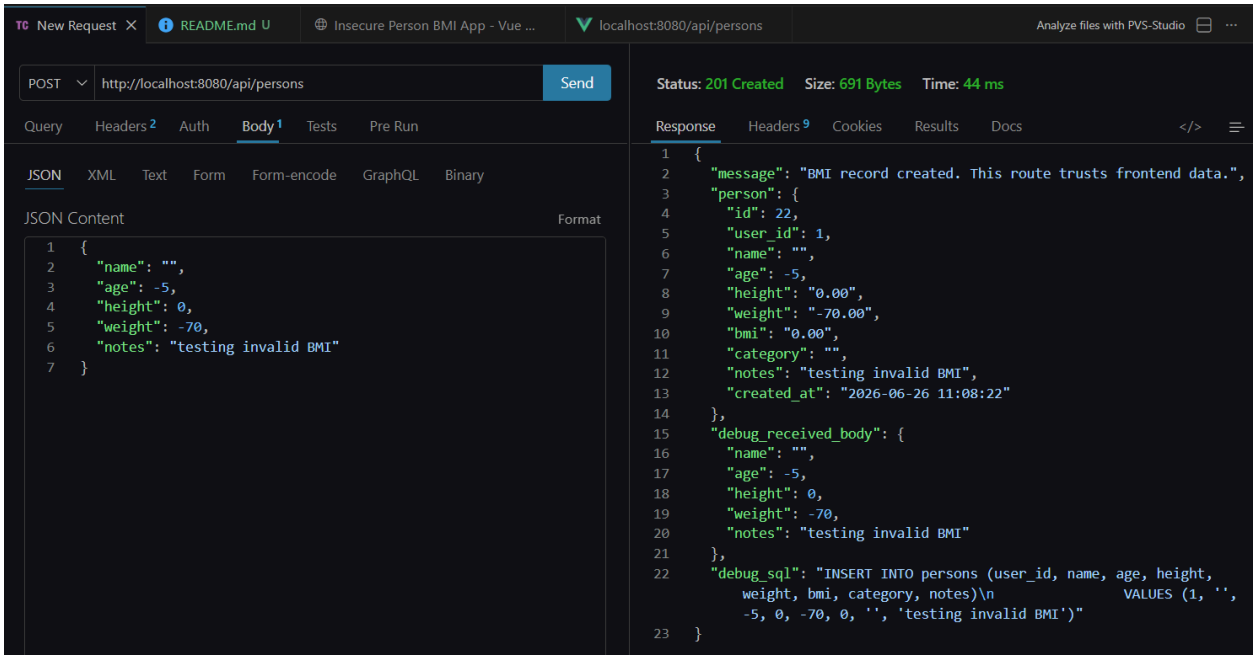
No, because I can bypass the frontend UI using Thunder Client.

- Can the same request be sent using Postman?

Yes, anyone can intercept and send raw requests to the API.

- Where should the validation be enforced?

Backend, because frontend validation can be bypassed.



Test 2: Access another user's BMI record

- Did the backend return another user's BMI record?

Yes, changing the ID in the URL leaked other people's data.

- Is login alone enough?

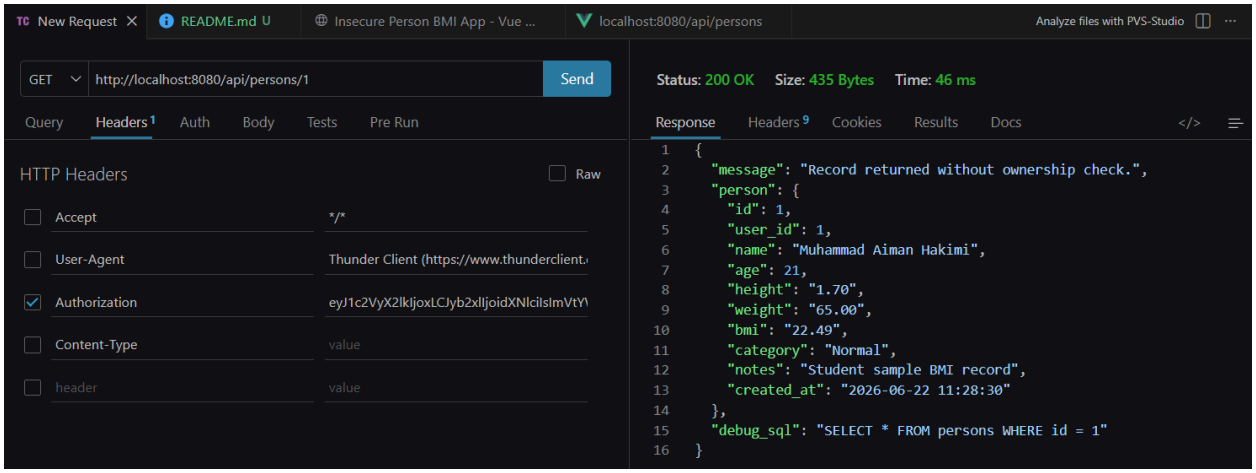
No, it is just authentication but no authorization.

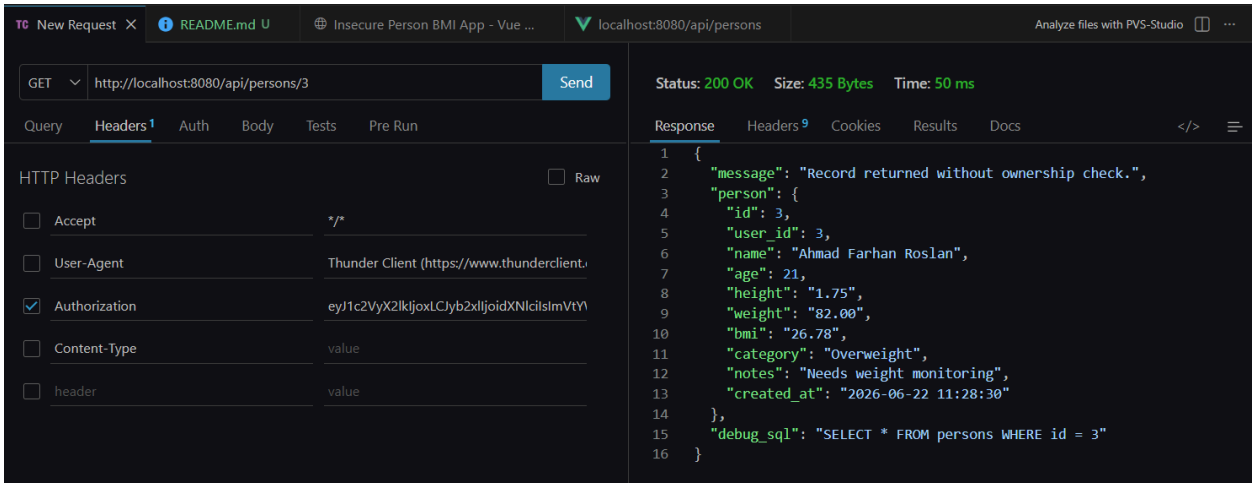
- What should the backend compare?

It must match the user_id from the JWT with the user_id on the database.

- Should staff and admin have different access?

Yes, staff and admin roles need to have different access to monitor all records.





Test 3: Access staff route as normal user

- Did the backend block the request?

No, a normal user token could access the restricted endpoints.

- Which roles should be allowed?

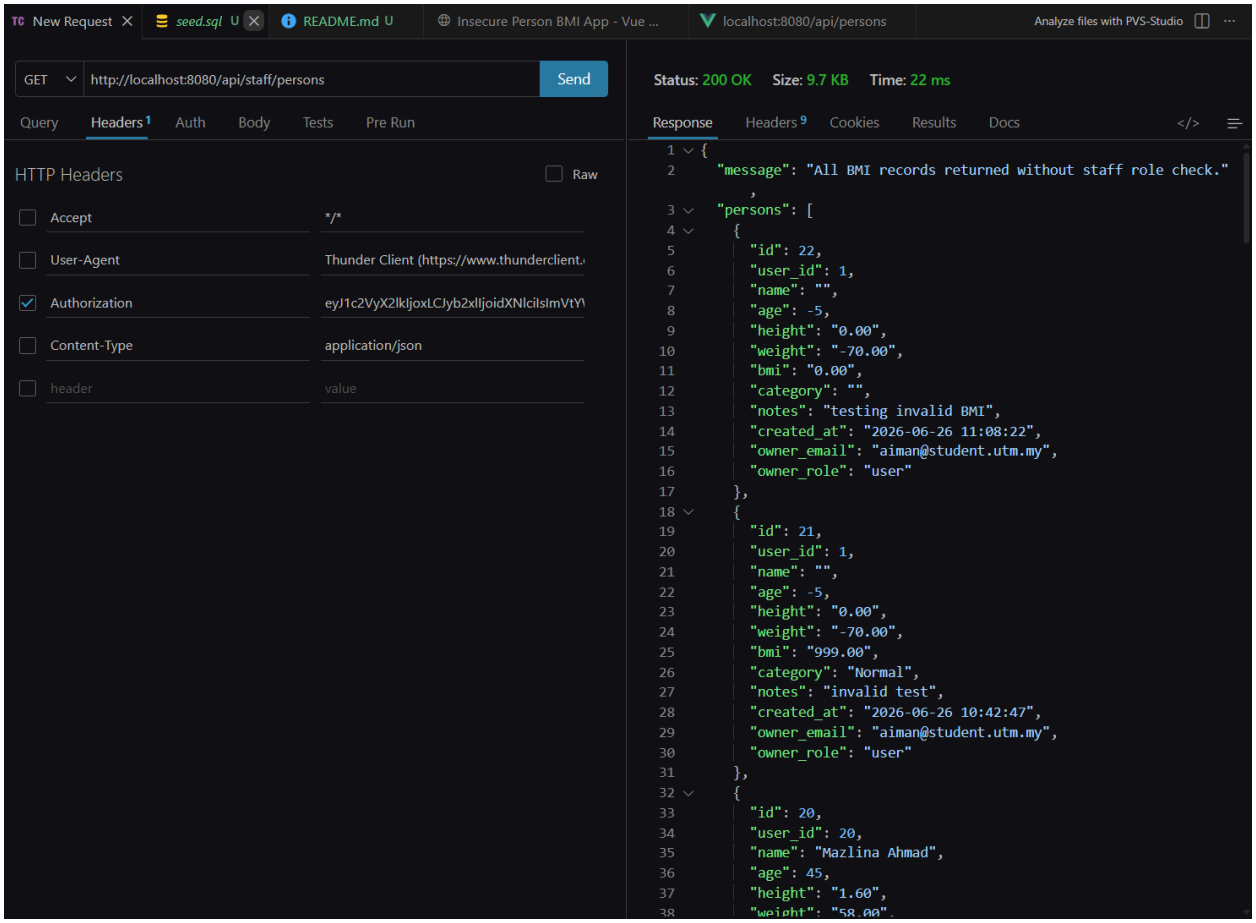
Only users with a role value of 'staff' or 'admin'.

- Should checking be done in frontend route guard only?

No, attackers don't use the frontend browser to make API requests.

- Where must the final decision happen?

In the backend routing middleware.



Test 4: Access admin route as normal user

- Did the backend accept user_id?

Yes.

- Did the backend accept role?

Yes.

- Did the backend accept bmi and category from the frontend?

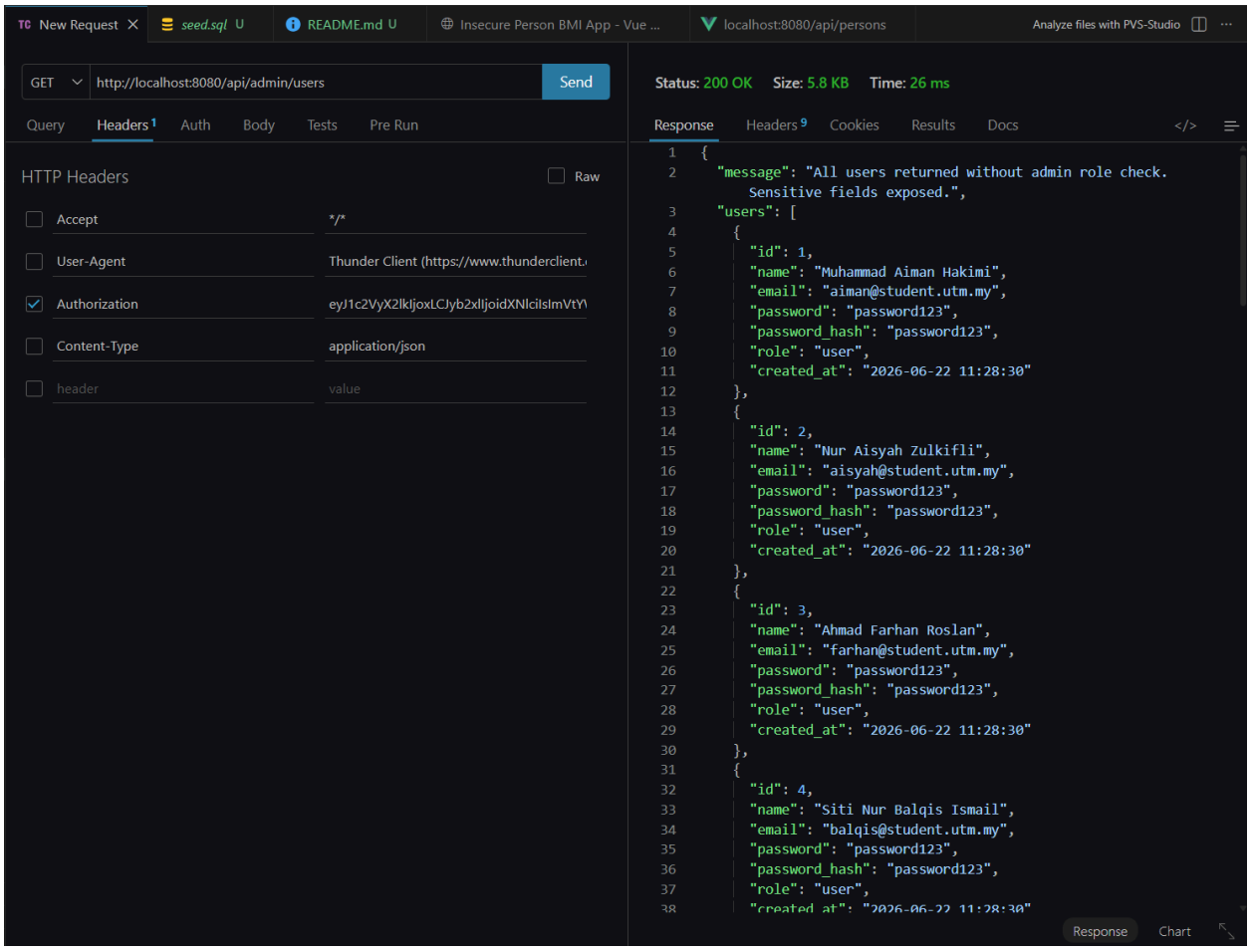
Yes, the database blindly updated them all.

- Which fields should be allowed?

Only name, age, height, weight, and notes.

- Which fields should be calculated or controlled by backend?

Structural fields like user_id and role, plus calculations like bmi and category.



Test 5: Submit XSS payload in notes

- Does the API return password_hash?

Yes, it was fully visible in the JSON response payload.

- Does the frontend really need this data?

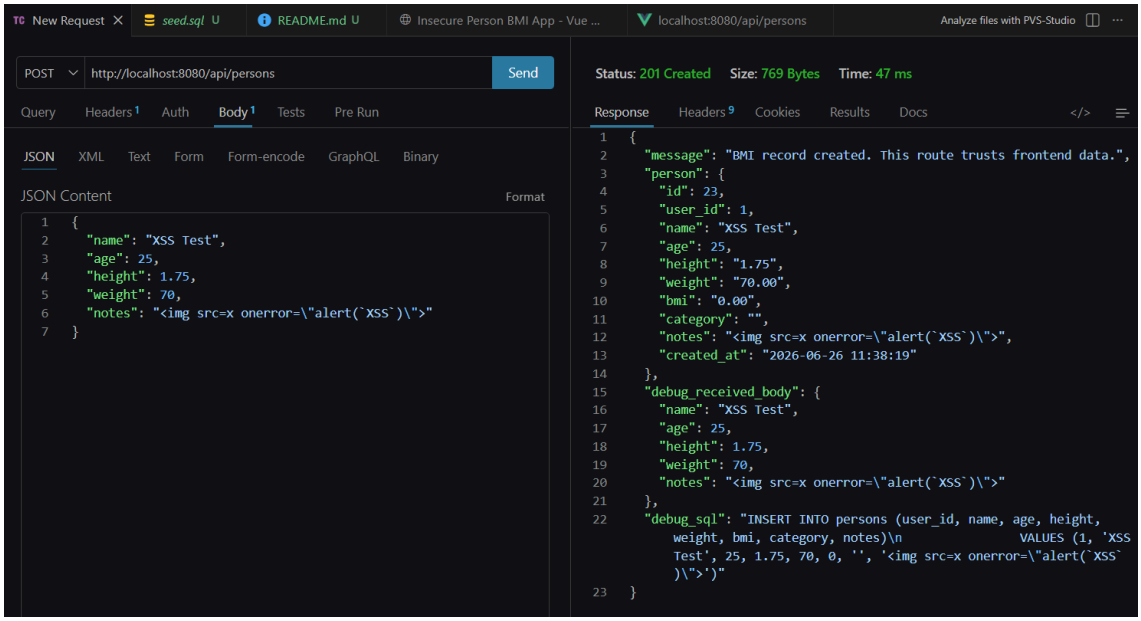
No, the client UI never needs to know password hashes.

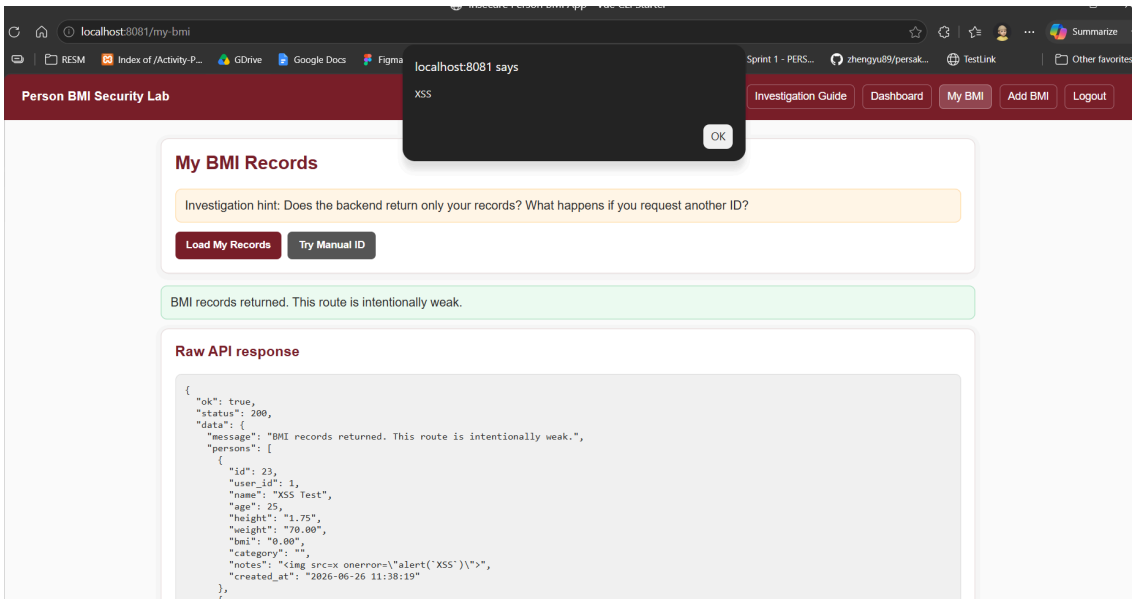
- What fields should be returned?

Only safe profile data: id, name, email, and role.

- How can SQL SELECT be improved?

Stop using SELECT * and explicitly write out the safe columns.





Test 6: Check API response

- Did the alert execute?

Yes, the script payload popped up in the browser window.

- Is the frontend using innerHTML or v-html?

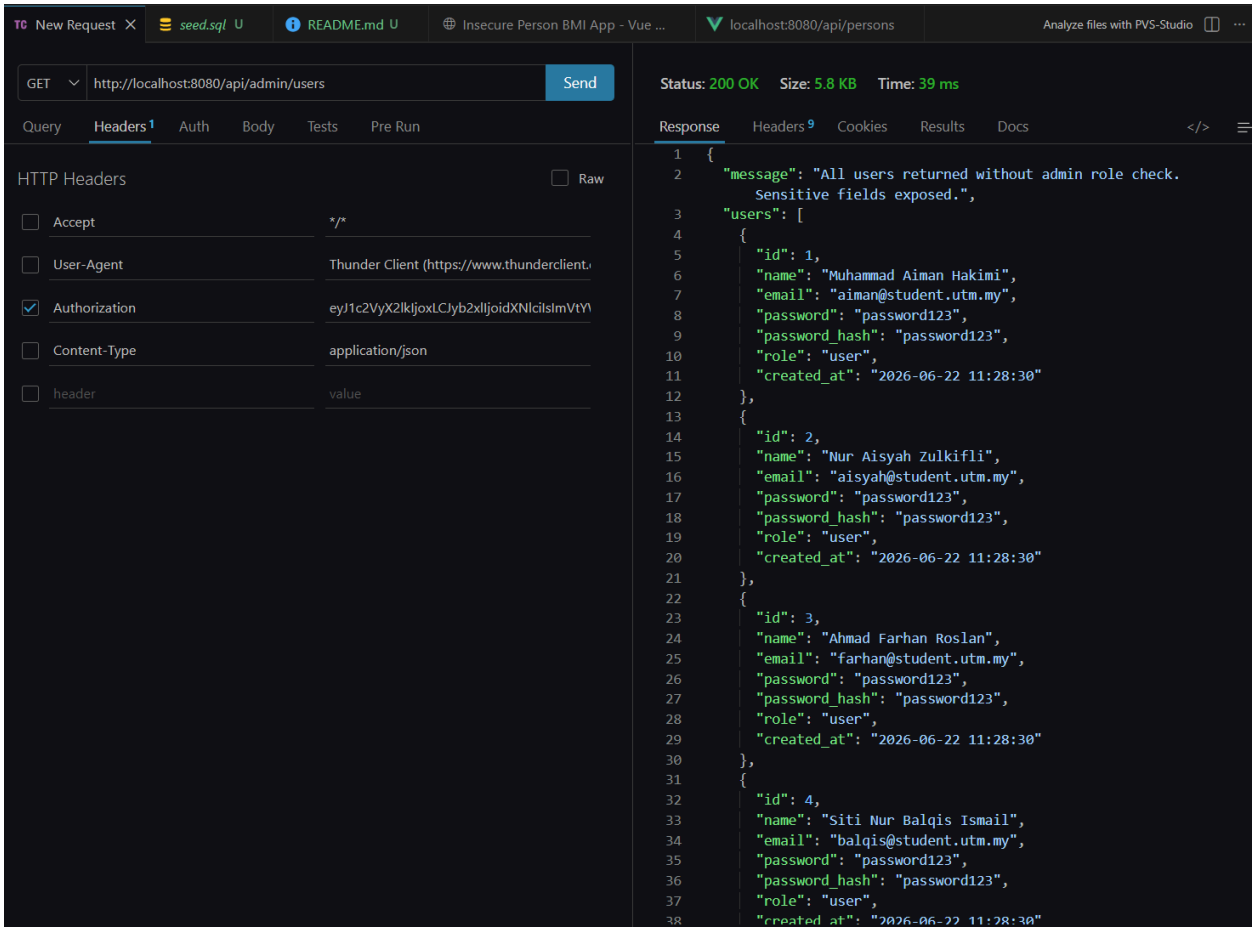
Yes, it used unsafe binding instead of text escaping.

- What is safer for displaying normal text?

Double curly braces {{ }} text interpolation or textContent.

- Should user-generated content be rendered as HTML?

No, never render raw user input as live HTML.



Test 7: Try SQL Injection input

- Did the SQL query behave unexpectedly?

Yes, inputting quotes broke out of the string and changed the logic.

- Was login bypassed?

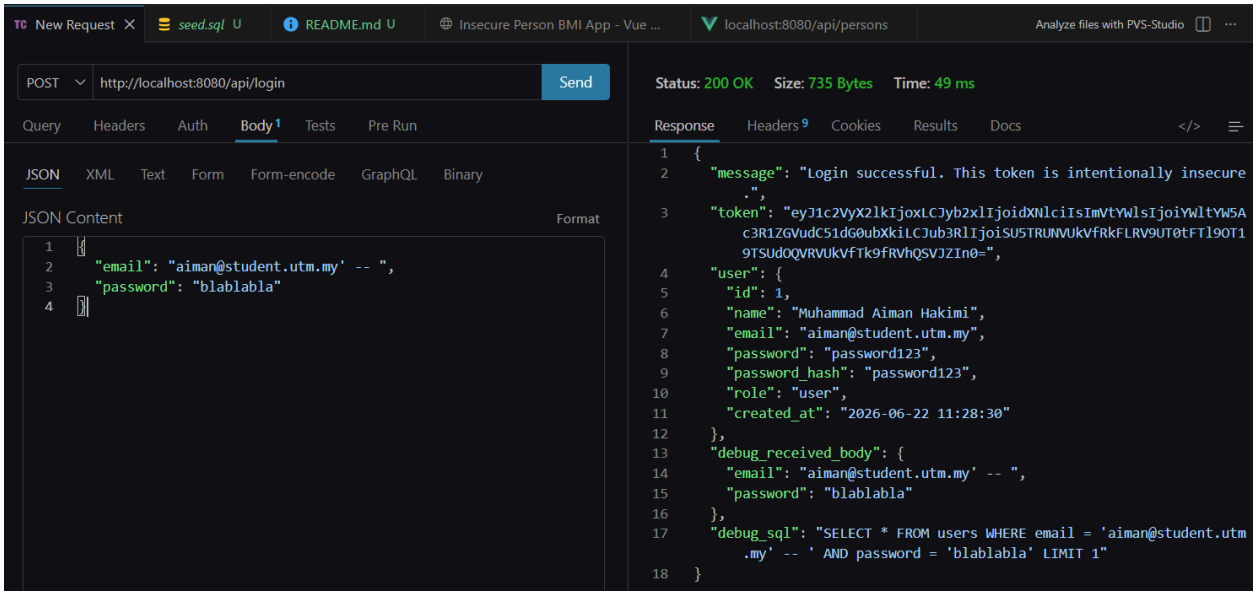
Yes, the payload successfully manipulated the query parameters.

- Was extra data returned?

Yes, it fetched records without validating real credentials.

- Is the SQL query using string concatenation?

Yes, it directly glued variables into the raw query string.



Test 8: Try to update protected fields